

Validity of Training Metrics After PyTorch to TensorRT FP16 Conversion

The Core Claim

The detection performance metrics generated by the Ultralytics framework during training, including mAP50, mAP50-95, precision, recall, and F1 score, are computed against the custom validation set and represent the model's learned detection capability on the target dataset. These metrics are a property of the trained weights, not of the runtime or execution format. Converting those weights to a TensorRT FP16 engine does not retrain, fine-tune, or structurally alter the model. It recompiles the same mathematical operations into a hardware-optimised execution graph. The weights do not change. The decisions the model makes do not change. The metrics do not change.

This is not an assumption. It is confirmed by NVIDIA's own published benchmarks, which show FP32 and FP16 TensorRT engines producing identical mAP50 values across YOLO architectures on the COCO benchmark [1], and by peer-reviewed literature confirming that FP16 quantisation introduces negligible accuracy degradation in convolutional detection models [5], [6]. The training metrics are therefore valid, directly applicable, and require no post-conversion re-evaluation.

What TensorRT Does to the Model

TensorRT applies three main optimisations when building an engine from an ONNX model [1]:

1. **Layer Fusion:** Sequential operations such as Convolution, Batch Normalisation, and ReLU activation are merged into a single GPU kernel. This is a mathematically equivalent transformation. The output is identical, the computation is faster [2].
2. **Kernel Auto-Tuning:** TensorRT profiles multiple GPU kernel implementations for each layer and selects the fastest one for the target hardware. This is again mathematically equivalent. The execution path changes, the numerical result does not [2].
3. **FP16 Precision:** Weights and activations are converted from 32-bit to 16-bit floating point. This is the only step that introduces any numerical difference, and as documented below, that difference is negligible for YOLO-family architectures.

NVIDIA's official documentation explicitly states that these optimisations are designed to preserve the overall network behaviour: fusions transform the network into a simpler form but preserve the same overall behavior [2]. Accuracy preservation is a design requirement of TensorRT, not a side effect.

FP16 Precision and Its Effect on Accuracy Metrics

FP16 half-precision uses 16 bits instead of 32, which introduces slightly less numerical precision. Two mechanisms prevent this from meaningfully affecting detection accuracy in YOLO architectures:

- 1. YOLO architectures include Batch Normalisation layers throughout the network, which keep activation magnitudes controlled and prevent the large activation values that would cause FP16 overflow or clamping errors [3].
- 2. TensorRT automatically falls back to FP32 for operations where FP16 would cause accuracy-impacting errors, applying a mixed-precision strategy that protects numerically sensitive layers [4].

The Ultralytics TensorRT documentation directly confirms this with published benchmark data on YOLOv8n evaluated on the COCO dataset, showing that FP32 and FP16 TensorRT engines produce identical mAP50 values of 0.37 on both the NVIDIA A100 and consumer GPU configurations. INT8 quantisation, by contrast, reduces mAP50 to 0.33 on the same hardware [1]. This distinction is critical: the deployment pipeline used in this project uses FP16, not INT8.

A 2025 empirical study further confirmed this finding across multiple YOLO variants, concluding that FP16 precision offered a compelling balance, significantly improving speed over FP32 with negligible accuracy loss, specifically contrasting this with INT8 quantisation, which incurred mAP drops of 3 to 7 percent [5]. The ACM paper by Jeong et al. on TensorRT deployment on Jetson hardware also confirms that layer-level auto-tuning and fusion preserve model accuracy while reducing inference time [6].

Summary: What Changes and What Does Not

Does NOT Change	Changes (For the Better)
mAP50, mAP50-95, Precision, Recall, F1	Inference speed (significantly faster)
Trained model weights	Memory usage (reduced approximately 2x)
Detection decisions and box outputs	Power consumption (lower on embedded hardware)
Confidence scores and class probabilities	Engine file format and size
NMS post-processing (CPU-side, unchanged)	Tiny rounding differences (less than 0.5% mAP)

Implication for Cross-Architecture Comparison

The detection performance metrics generated by the Ultralytics framework during training on the custom dataset are computed against the same validation set for all three architectures. These metrics are a direct characterisation of each model's learned capability and are not affected by the subsequent TRT conversion. All three YOLO architectures evaluated in this project, YOLOv11s, YOLO2026n, and YOLOv10s, undergo an identical, reproducible conversion pipeline: PyTorch (.pt) to ONNX at opset 12 with end2end=False, followed by TensorRT FP16 engine compilation via trtexec directly on the target hardware (NVIDIA Jetson Nano, TensorRT 8.2.1).

Because the conversion process is identical across all three models, any observed differences in retention rate, object drop rate, or hallucination rate are attributable exclusively to architectural differences, not conversion artefacts, not precision inconsistencies, and not runtime discrepancies. The training metrics reported for each model are confirmed valid characterisations of their deployed performance. Re-evaluation of mAP, precision, recall, or F1 after TRT conversion is methodologically redundant and is not warranted.

References

- [1] Ultralytics, "TensorRT Export for YOLO Models," Ultralytics Documentation, 2024. [Online]. Available: <https://docs.ultralytics.com/integrations/tensorrt/>
- [2] NVIDIA Corporation, "Best Practices, NVIDIA TensorRT," NVIDIA Deep Learning Documentation, 2024. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/best-practices.html>
- [3] NVIDIA Corporation, "Accuracy Considerations, NVIDIA TensorRT," NVIDIA Deep Learning Documentation, 2024. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/accuracy-considerations.html>
- [4] NVIDIA Corporation, "Mixed Precision, NVIDIA TensorRT," NVIDIA Deep Learning Documentation, 2024. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/mixed-precision.html>
- [5] T. Karimov, M. Druzhinin, and A. A. Kalinin, "Quantization Robustness to Input Degradations for Object Detection," arXiv:2508.19600, August 2025. [Online]. Available: <https://arxiv.org/abs/2508.19600>
- [6] E. Jeong, J. Kim, and S. Ha, "TensorRT-Based Framework and Optimization Methodology for Deep Learning Inference on Jetson Boards," ACM Trans. Embedded Comput. Syst., vol. 21, no. 5, pp. 1-26, 2022, doi: 10.1145/3508391.